

Decision-OS V9.1: Impact-Weighted Release

From Continuous-Pass Gates to Condition-Bound Judgment Reuse

Shinichi Nagata

Abstract

Decision-OS V9 defines a release procedure for high-impact systems operating under public repetition. The core problem is not a lack of intelligence, but failure amplification caused by (i) hindsight overwrite, (ii) responsibility laundering, and (iii) repeated exposure with independence breakdown (the *Public Tube*). V9 addresses this by fixing *As-of* (temporal fidelity), *Seat* (decision rights and accountability), and *Release* (impact-weighted continuous pass). Time V2 separates evaluation into a Playback-only lane bound to an As-of Pack and a Forward-only update lane expressed as explicit deltas, preventing post-hoc narrative edits of the past. This update further extends V9 from a review-and-release protocol into a reusable judgment-state protocol. Without Judgment Compression, past decisions remain costly to reuse because each later decision must reconstruct the full As-of context, dependencies, and residue from scratch. DFR separates forward deltas from preserved residue, and Judgment Compression stores eligible residue as

(outcome, residue_key, condition).

A compressed judgment remains reusable only while its condition holds; if the condition is invalidated, stale, or unverified, the state defaults to DELAY and returns to recheck or DFR. For public deployment, V9 treats repetition as a first-class risk factor: release is granted only after sustained passing over an observation window that scales with impact and exposure. We also describe interaction primitives (CAP and DFR) and a disagreement-based method (MMAR) to extract invariants and residue candidates without collapsing uncertainty into persuasive coherence. Overall, V9 shifts the criterion from “smart answers” to procedures that remain stable under audit, hindsight, repeated public trials, and condition-bound reuse.

Index Terms

Impact-weighted release, temporal fidelity (As-of), accountability (Seat), Public Tube, continuous pass, Time V2, CAP/DFR, MMAR, Judgment Compression, reusable gate state

I. INTRODUCTION

Safety is not the ability to “get the right answer.” Safety is the ability to *stop*, *delay*, and *roll back* when the world flips.

Decision-OS V8 formalized this stance as *Time-Tube control*: safety is enforced over a line (trajectory), not by claiming one-shot correctness at a point. However, real failures are often amplified not by a single mistake, but by how evaluation and responsibility are handled *after the fact*. Time does not only amplify value; through narrativization and hindsight, it can amplify exemption, condemnation, and the erosion of accountability.

Decision-OS V9 does not aim to predict the future. V9 provides an operational procedure that is harder to break: it fixes the as-of moment when a decision was made, separates improvements into a future delta lane, and defines *Release* as an impact-weighted *continuous pass* rather than one-shot correctness.

This update further extends V9 from a review-and-release protocol into a reusable judgment-state protocol. Human decision capacity is bounded to a small number of interacting layers; without compression, multi-layer judgment becomes non-operational rather than merely inefficient. As-of Fixation preserves the temporal boundary of the original decision, DFR extracts the forward delta and preserved residue, and Judgment Compression converts that residue into a reusable gate state for future decisions. The compressed state is later stored in the form

(outcome, residue_key, condition),

where the outcome is normalized into PASS, DELAY, or BLOCK.

A. Lineage: from “Decision” to “Impact-Weighted Release”

The series position can be summarized as follows:

- V4: Decision as form (an entry OS for seeing structure).
- V5: Protection-of-life gates (stop / verify / confirmation).
- V6: PIC (canonicalization; phase-invariant merge).
- V7: Aspire Intelligence (recursion-aware operational framing).

- V8: Time-Tube Control (point $\rightarrow$ trajectory; intervention over time).
- V9: Impact-Weighted Release (Time V2; Public Tube + Hindsight Control).

V8 focuses on internal safety: intervention over a trajectory. V9 canonicalizes this into external operational rules that remain valid under public release and repeated exposure. In this update, V9 also inherits from V8 the view that irreducible mismatch may act as a control-relevant signal; however, in V9 it is treated only as a *residue candidate signal*, not as a probability estimate or as the main object of the protocol.

B. As-of fixation and update-lane separation

V9 introduces *As-of Fixation*: a decision is evaluated against the bundle of information, constraints, and objectives available at the time—the *As-of Pack*.

V9 also introduces *Update Lane Separation*:

- **Playback-only lane**: explanations are constrained to the As-of Pack.
- **Forward-only update lane**: improvements are proposed only as future operational deltas.

This separation prevents hindsight from “beating the past” and destroying learning. It also prevents responsibility laundering via post-hoc reframing, where time is used to justify exemption or to weaponize condemnation.

C. Impact-weighted release under public repetition

V9 defines *Release* as a gate-based condition, not a narrative justification. Release is granted only after *sustained passing*, with longer observation windows for higher-impact decisions. This accumulates institutional defense cost against the *Public Tube* phenomenon: even small success probabilities can eventually “hit” through repetition, large n , and independence breakdown.

In this version, probability-related variables such as p or P are not treated as the main axis of V9. They remain auxiliary inputs for estimating exposure, reach, execution capability, or release pressure. The central object is the judgment state that remains reusable after As-of review, DFR, residue filtering, and Judgment Compression.

In short, V9 shifts the center of gravity from “smart answers” to procedures that do not break. Safety is not intelligence—it is a gate.

D. Minimal structure: three outcomes and three fixations

V9 commits to a minimal structure that all subsequent chapters extend without breaking.

a) Outcomes (judgment lane):

- **PASS** (proceed).
- **DELAY** (do not decide yet; recheck window applies).
- **BLOCK** (do not proceed).

b) Fixations (non-negotiable anchors):

- **As-of**: fix temporal fidelity (As-of Pack).
- **Seat**: fix responsibility and decision rights (Decision Owner / Auditor).
- **Release**: publish only via impact-proportional observation windows and continuous pass.

c) *Reusable judgment state*: The V9.1 extension adds a fourth operational layer without replacing the three fixations: preserved residue may be compressed into a reusable judgment state. A residue is preserved only when it changes branching, reversibility, Seat assignment, dependency structure, or non-reconstructability. When the stored condition no longer holds, the compressed judgment does not remain PASS by default; it is downshifted to DELAY and must be rechecked through DFR before reuse.

E. Contributions

This paper contributes an operational constitution for high-stakes decisions under public release:

- **Time V2 procedure**: As-of Pack $\rightarrow$ Playback-only $\rightarrow$ Forward-only update (fixed protocol).
- **Hindsight control**: review as temporal-fidelity preservation plus exactly one future delta, without character judgment.
- **Public Tube framing**: repeated exposure and independence breakdown as first-class risk factors for release.
- **Impact-weighted continuous pass**: release scales with impact and requires sustained passing, not one-shot success.
- **Judgment Compression**: preserved residue is converted into a reusable gate state stored as (outcome, residue_key, condition).

This motivates Time V2 as a procedural defense against hindsight bias [1].

II. OVERVIEW

This chapter summarizes the core claim of Decision-OS V9:

Release is not a narrative.

Release is a procedure.

The procedure must remain valid under hindsight, repetition, and public exposure.

For public-facing systems, documenting intended use, limits, and evaluation context is part of safe release [2].

V9 defines the minimal operational structure that survives these pressures by fixing (i) temporal fidelity (*As-of*), (ii) responsibility (*Seat*), and (iii) publication conditions (*Release*). In this update, V9 is also made explicit as a reusable judgment-state protocol: after *As-of* review, preserved residue is separated from future deltas and may be compressed into a reusable gate state for later decisions.

A. Problem: why “good reasoning” still fails in public

Even when a decision is made with careful reasoning, public reality introduces three failure amplifiers:

- **Hindsight overwrite:** post-hoc narratives replace the original constraints and objectives, destroying learning.
- **Responsibility laundering:** explanations drift into blame or exemption, moving responsibility away from the decision owner.
- **Repetition amplification:** small probabilities “hit” under large n , especially when independence breaks.

These amplifiers are not solved by smarter answers. They are solved by constraining what can change, and where it is allowed to change.

B. V9 in one page: *As-of*, *Seat*, *Release*

V9 is built from three anchors.

a) (1) *As-of: temporal fidelity as a hard constraint:* A decision must be reviewed against the information and constraints available at the time of execution. V9 packages this as an *As-of Pack* and binds explanations to *Playback-only*. Improvements are not allowed to “edit the past”; they must be expressed as *forward-only deltas*.

b) (2) *Seat: responsibility and decision rights are fixed before evaluation:* V9 treats responsibility assignment as a pre-evaluation gate. A review must first determine who holds the *decision seat* (Decision Owner) and who audits (Auditor). Without this, evaluation becomes either scapegoating or absolution, both of which destroy operational learning.

c) (3) *Release: impact-weighted continuous pass:* V9 defines release as an operational gate that scales with impact. Higher impact requires longer observation windows and repeated passing. Release is not granted by one-shot success.

d) *Operational bridge: DFR and Judgment Compression:* Between review and reuse, V9 inserts a fixed bridge: *DFR* separates what remains valid from what must move into the future-delta lane, while preserving the residue that should not be erased. Only preserved residue that changes branching, reversibility, *Seat* assignment, dependency structure, or non-reconstructability is eligible for *Judgment Compression*. The compressed form is later stored as

(outcome, residue_key, condition),

where the outcome is normalized into PASS, DELAY, or BLOCK.

C. Public Tube: repeated exposure as a first-class risk factor

V9 treats public release as a *tube* that accumulates attempts, viewers, and opportunities for misuse. Even when a single attempt has a low probability of harm, repeated exposure increases aggregate risk. In addition, independence breaks in public: copying, re-sharing, and coordinated trials make outcomes more correlated. This accumulation and union effect motivates the release gate defined later in Section III-J.

Therefore, release must account for:

- repeated trials (n growth),
- independence breakdown (correlation),
- and impact magnitude (harm scale).

In this sense, V9 does not treat probability-related terms such as p or P as the main object of the protocol. They remain auxiliary inputs for estimating exposure, reach, execution capability, or release pressure. The main object is the judgment state that survives hindsight review and remains reusable under public repetition.

D. Outputs: three outcomes plus explicit recheck

V9 standardizes judgment into three outcomes:

- **PASS:** proceed.
- **DELAY:** do not decide yet; set an explicit *until* time and recheck conditions.
- **BLOCK:** do not proceed.

DELAY is not indecision. It is a controlled safety state that preserves agency while refusing premature commitment.

When a compressed judgment is reused, the stored condition must still hold. If the condition has become invalid, stale, or unverified, the prior state does not remain PASS by default; it is downshifted to DELAY and must be rechecked through the DFR path.

E. Roadmap of the paper

The remainder of this paper proceeds as follows:

- Section III defines the fixed protocol (As-of Pack, CAP/DFR, release gates, and reusable judgment states).
- Section IV describes the system architecture and canonical merge constraints.
- Section V details Time V2 and the anti-hindsight mechanism.
- Section VI covers interaction patterns and MMAR-style structure extraction.
- Section VII describes multi-party confirmation and Seat control under pressure.

III. PROTOCOL

A. Protocol purpose: fixed review before reuse

This chapter defines the fixed operational protocol of Decision-OS V9. The protocol is designed to survive (i) hindsight pressure, (ii) responsibility drift, and (iii) repeated public exposure.

V9 is intentionally procedural: it constrains *what* may change and *where* it may change. All evaluations are bound to an As-of Pack, and all improvements are expressed only as forward deltas. In this update, the protocol also specifies when preserved residue may be compressed into a reusable judgment state.

B. As-of Pack: what is fixed

An *As-of Pack* is the minimal bundle that must be fixed before any evaluation:

- **Timestamp (As-of):** when the decision was executed.
- **Context:** constraints, resources, environment, and capability ceiling at the time.
- **Objective:** the objective function used at the time, including safety constraints.
- **Action:** what was actually done, not what was merely intended.
- **Evidence:** the observable inputs available at the time, such as logs, screenshots, records, or archived artifacts.

V9 binds explanations to the As-of Pack (*Playback-only*). Hindsight is permitted only as *forward-only deltas* through DFR (Section III-E).

C. Pre-evaluation gates: Seat, deadline, probability

Before judging correctness or quality, V9 fixes three gates.

a) Seat (responsibility): Assign roles explicitly:

- **Decision Owner:** holds the decision rights and responsibility for the action.
- **Auditor:** reviews the action and proposes deltas; does not take the seat.

b) Time horizon (deadline): If a decision includes a deadline or time sensitivity, record the horizon explicitly, such as hours, days, or weeks. This prevents “deadline drift” in post-hoc narratives.

c) Probability (uncertainty): If the decision implicitly assumes a probability p , record whether p was (i) estimated, (ii) bounded, or (iii) unknown. Unknown is a valid state; it is handled via **DELAY** (Section III-I).

Probability-related terms such as p or P are auxiliary inputs in V9. They may help estimate uncertainty, reach, execution capability, or release pressure, but they do not define the main protocol object. The main object is the judgment state that survives As-of review, DFR, residue filtering, and Judgment Compression.

D. CAP v1: Claim, Dependencies, Alert-if, Recheck, Hedge

V9 uses a minimal claim protocol to prevent post-hoc story edits.

a) *CAP record*: Each decision claim is expressed as:

- **Claim**: the actionable statement, namely what is asserted.
- **Dependencies**: which assumptions must hold.
- **Alert-if**: observable signals that invalidate the claim.
- **Recheck**: when and how the claim will be revisited.
- **Hedge**: the safety action if the claim weakens, such as stop, downshift, rollback, or partial freeze.

b) *Constraint*: A CAP record must be written in As-of terms. New information cannot be injected into the Claim retroactively; it must appear as a forward delta.

E. DFR v1: As-of $\rightarrow$ Delta $\rightarrow$ Residue $\rightarrow$ Next step

V9 standardizes review into a forward-only form. This prevents two failure modes: (i) moral condemnation that destroys learning, and (ii) hindsight absolution that launders responsibility.

a) *DFR steps*:

- 1) **As-of Playback**: restate the decision under the As-of Pack, with no new facts.
- 2) **Delta**: specify exactly what would be changed going forward, such as a procedure, threshold, check, or guard.
- 3) **Residue**: specify what remains valid: invariant structure that should not be edited away.
- 4) **Next step**: choose one operational update to implement next, as a single change rather than a redesign.

b) *One-change rule*: A single review produces at most one operational update. If more changes are desired, they must be queued as separate deltas with separate rechecks.

F. Residue Criterion: what may be preserved

Operationally, V9 treats residue as the deployment-level admission layer for the V8 residue form $R_{\text{res}}(x) := \text{Canon}(R(\Delta(x)))$, while replacing V8's deferred residue-admission question with explicit preservation criteria.

Not every remaining statement after DFR becomes reusable residue. V9 preserves residue only when it changes at least one commitment-relevant operational structure.

A residue is preserved only when it affects at least one of the following:

- 1) **Branching**: it changes which future paths remain open or closed.
- 2) **Reversibility**: it changes whether rollback, exit, or repair remains possible.
- 3) **Seat**: it changes responsibility assignment or decision rights.
- 4) **Dependency**: it changes an assumption, prerequisite, external reliance, or trigger condition.
- 5) **Non-reconstructability**: it captures structure that cannot be reconstructed later from local inference or a smoother narrative.

Equivalently: a residue is preserved only when it changes branching, reversibility, Seat assignment, dependency structure, or non-reconstructability.

Irreducible mismatch may serve as a *residue candidate signal*, especially when a reconstructed explanation fails to recover the original structure. However, irreducible mismatch is not a probability estimate and is not the main object of V9. It only indicates that a residue candidate should be examined under the criterion above.

G. Judgment Compression: residue to reusable gate state

Judgment Compression converts preserved residue into a reusable gate state. Judgment Compression is not summarization; it is a reduction into executable gate states. Unlike knowledge storage, it encodes decision outcomes under condition-bound operational reuse, not descriptive recall. It does not rewrite the As-of Pack and does not replace DFR. It only compresses the residue that survives DFR and the Residue Criterion into a form that can be reused in later decisions.

The compressed state is stored as:

(outcome, residue_key, condition)

where:

- **outcome** is normalized into PASS, DELAY, or BLOCK;
- **residue_key** identifies the preserved structure;
- **condition** specifies when the compressed judgment remains valid.

a) *Condition invalidation rule*: When the stored condition no longer holds, the compressed judgment does not remain PASS by default. It is downshifted to DELAY and must be rechecked through DFR before reuse.

This rule prevents stale judgments from becoming hidden permanent permissions. A reusable judgment state is reusable only while its condition remains valid.

Judgment Compression Pipeline under As-of Review

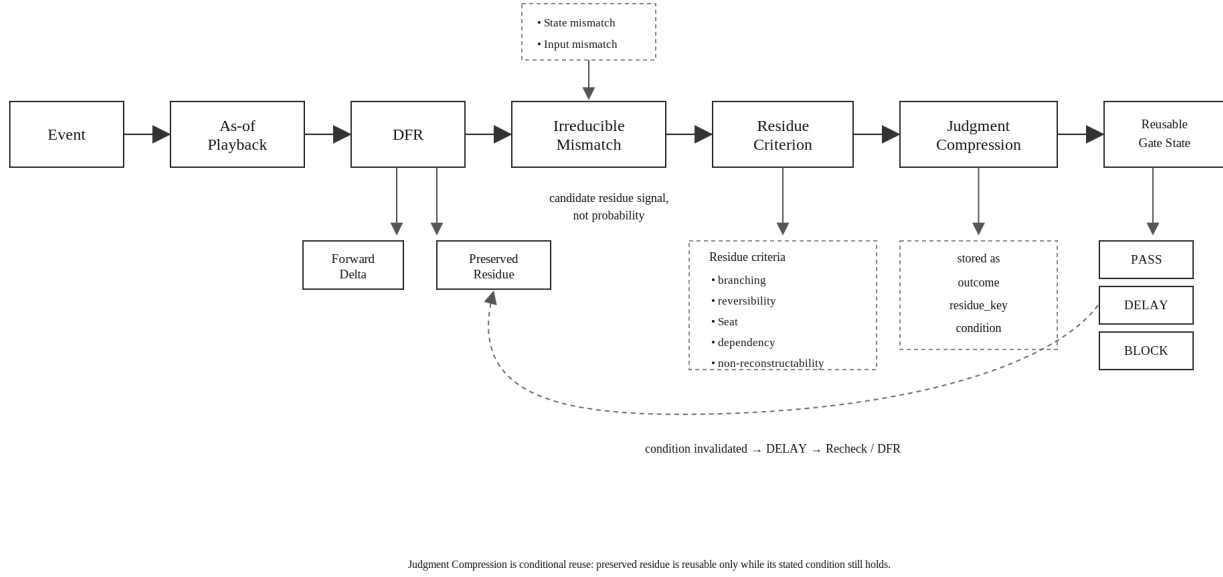


Fig. 1. Judgment Compression Pipeline under As-of review. DFR separates a forward delta from preserved residue, while irreducible mismatch acts as a candidate residue signal rather than a probability estimate. Residue is filtered by branching, reversibility, Seat, dependency, and non-reconstructability before being compressed into a reusable gate state. The compressed judgment is stored as (outcome, residue_key, condition); if the condition is invalidated, it defaults to DELAY and returns to recheck or DFR.

H. Judgment lane: PASS, DELAY, BLOCK

V9 constrains outcomes to three states:

- **PASS:** proceed under the current protocol.
- **DELAY:** do not decide yet; set an explicit *until* time and recheck triggers.
- **BLOCK:** do not proceed; the action is rejected under current constraints.

These states apply both to private decisions and to public release decisions. The difference is that public release is additionally constrained by the Release gate (Section III-J).

I. DELAY: controlled non-commitment with explicit until

DELAY is not indecision; it is a safety state that preserves agency.

A DELAY record must specify:

- **until:** the earliest time the decision may be revisited;
- **recheck conditions:** what must be observed to revisit;
- **hedge while waiting:** what safety posture is enforced during the delay.

If a decision is high impact and uncertainty is not bounded, DELAY is the default safe state. If a compressed judgment is reused and its condition is stale, broken, or unverified, DELAY is also the default state.

J. Release gate: impact-weighted continuous pass

V9 treats release as a gate that scales with impact and exposure.

We operationalize impact as a minimal product of externality, responsibility, and power:

$$I(a) := E(a) \cdot R(a) \cdot P(a).$$

a) *Impact weighting:* Let I denote impact magnitude, such as harm scale if misused or irreversible cost if wrong. Higher I requires longer observation windows and stricter passing conditions.

b) *Continuous pass:* Release is granted only if the system remains in PASS across a required observation window. In operational terms: the process must pass *continuously*, not once.

Release Gate Structure under Impact-Weighted Observation

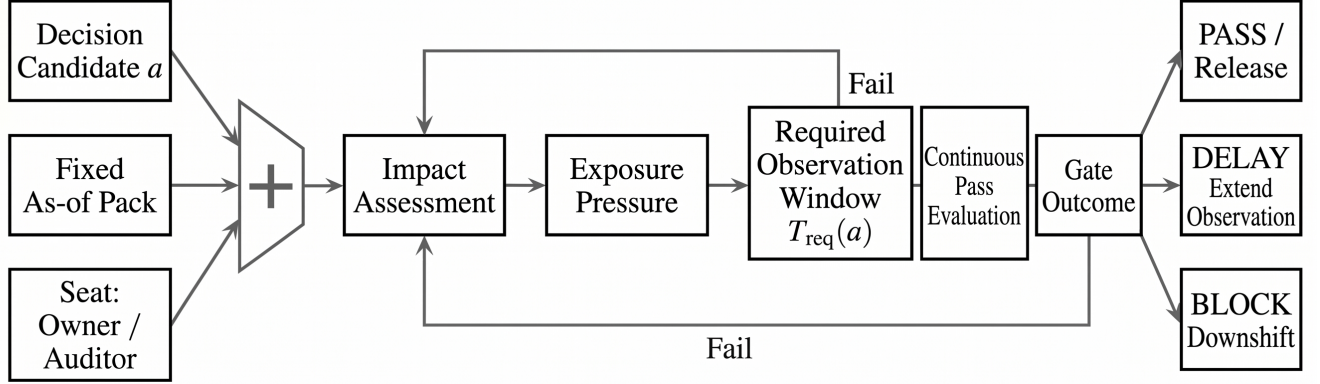


Fig. 2. Release Gate Structure under impact-weighted observation. The gate normalizes release decisions into PASS, DELAY, and BLOCK: PASS proceeds to Release, DELAY extends observation and recheck, and BLOCK routes the action to Downshift.

c) Public Tube pressure: Public release increases attempt count n and often breaks independence via copying and coordination. Therefore, release must consider:

- growth of n over time,
- correlation and independence breakdown,
- and worst-case misuse paths.

When impact is high or independence is likely to break, the protocol must shift toward DELAY or BLOCK unless strong guards are present.

We align the release gate with established risk management practice for trustworthy AI [3].

K. Protocol summary

The V9 protocol can be executed as the following fixed sequence:

- 1) Fix the As-of Pack (Section III-B).
- 2) Apply pre-evaluation gates: Seat, deadline, probability or uncertainty (Section III-C).
- 3) Write CAP for the claim (Section III-D).
- 4) Choose outcome: PASS / DELAY / BLOCK (Section III-H).
- 5) If reviewing: run DFR and implement exactly one forward delta (Section III-E).
- 6) If residue remains: apply the Residue Criterion and, when eligible, compress it into a reusable gate state (Sections III-F–III-G).
- 7) If reusing a compressed judgment: verify its stored condition; if invalidated, default to DELAY and recheck through DFR.
- 8) If publishing: apply the impact-weighted continuous pass Release gate (Section III-J).

IV. ARCHITECTURE

This chapter describes the system architecture that makes the V9 protocol operational. The goal is not to build a large framework, but to define a minimal structure that remains stable under iteration, public release, reuse, and audit.

V9 models decision-making as state updates. Each module emits a *delta* that is merged into a global state via a canonical merge rule. This prevents order-dependent behavior and reduces accidental “story edits”.

In this update, the architecture also supports reusable judgment states. DFR separates forward deltas from preserved residue, and Judgment Compression stores eligible residue as a reusable gate state without rewriting the original As-of Pack.

A. Global state and module deltas

Let S be the global operational state. Each module produces an update ΔS_i . The system merges updates into a new state:

$$S' = \text{Canon}(S \sqcup \Delta S_1 \sqcup \Delta S_2 \sqcup \dots).$$

The merge operator $\sqcup$ is designed to be:

- **commutative** (order does not matter),
- **associative** (grouping does not matter),
- **idempotent** (reapplying the same delta changes nothing).

$\text{Canon}(\cdot)$ is used in the V6 sense: a commitment-bounded canonicalization rule that collapses only commitment-irrelevant distinctions while preserving distinctions that can change commitment, safety state, or audit outcome. This makes audits tractable without treating Canon as a full identity theory.

This merge rule applies to committed operational state. It does not mean that exploratory candidates, model disagreements, or residue candidates are collapsed immediately. Candidate structures may remain in a frontier until DFR, residue filtering, and commitment-side checks determine what should be preserved.

B. Core registries

V9 maintains a small set of registries, each append-only by default:

- **As-of Pack Registry:** records the fixed bundle for each decision instance.
- **Seat Registry:** assigns Decision Owner and Auditor before evaluation.
- **CAP Registry:** stores Claim, Dependencies, Alert-if, Recheck, Hedge.
- **DFR Review Log:** stores As-of Playback, Delta, Residue, Next step.
- **Residue Registry:** records preserved residue that passes the Residue Criterion.
- **Judgment State Registry:** stores compressed gate states in the form (outcome, residue_key, condition).
- **Release Gate Log:** records impact weighting, observation windows, and pass continuity.
- **Public Tube Signals:** tracks exposure growth and independence breakdown indicators.

The primary design rule is separation:

- explanation is bound to As-of (Playback-only),
- improvement is written only as a forward delta (Forward-only update),
- preserved residue is stored separately from both the As-of Pack and the forward delta,
- reusable judgment states are condition-bound and must be rechecked when their condition fails.

C. Residue and judgment-state records

A residue record is created only after DFR identifies structure that remains valid and the Residue Criterion confirms that it affects branching, reversibility, Seat assignment, dependency structure, or non-reconstructability.

A minimal residue record contains:

- **residue_key:** a stable identifier for the preserved structure;
- **source_asof:** the As-of Pack from which the residue was extracted;
- **criterion_hit:** branching, reversibility, Seat, dependency, non-reconstructability, or a combination of these;
- **evidence:** the evidence anchors supporting preservation;
- **reuse_scope:** where the residue may be reused.

A judgment-state record compresses the residue into a gate state:

$$(\text{outcome}, \text{residue_key}, \text{condition}).$$

Here, *outcome* is normalized into PASS, DELAY, or BLOCK; *residue_key* identifies the preserved structure; and *condition* specifies when the compressed judgment remains valid.

This record is not a new As-of Pack and does not overwrite the historical decision. It is a reusable state derived from preserved residue under explicit conditions. Judgment states that are downshifted to BLOCK after recheck may be archived under a deployment-defined retention policy; V9 specifies the validity and reuse rules, not a universal retention period.

D. Severity lattice and canonical merge rules

V9 reduces control to a three-level severity lattice:

$$\text{PASS} < \text{DELAY} < \text{BLOCK}.$$

When multiple modules propose outcomes, the merged outcome is the maximum severity. This ensures safety dominance under disagreement.

V9 also standardizes merges for common fields.

a) *Until time*: DELAY uses an explicit *until*. When multiple until values exist, the merged until is the maximum time, ensuring the longest required waiting window is respected.

b) *Evidence*: Evidence is merged as a set union. No evidence item is deleted by later explanations; removal requires an explicit audit action.

c) *Risk and impact*: Risk estimates and impact levels are merged conservatively. If two modules disagree, the higher risk or higher impact dominates the merged state.

d) *Condition validity*: When a reusable judgment state is invoked, its stored condition is checked before reuse. If multiple condition checks disagree, the conservative merge applies: an invalidated or unverified condition prevents automatic PASS reuse. The state defaults to DELAY until DFR or recheck restores a valid PASS or BLOCK classification.

E. Canonicalization: preventing story edits

The canonicalization function $\text{Canon}(\cdot)$ enforces invariants that protect temporal fidelity, responsibility assignment, and reusable-state validity:

- **As-of invariance**: As-of Pack fields cannot be modified by review outputs.
- **Seat invariance**: the decision seat cannot drift during review; Auditor cannot become Decision Owner mid-stream.
- **Delta isolation**: improvements must be written as forward deltas and cannot overwrite the As-of Playback.
- **Residue preservation**: residue that changes branching, reversibility, Seat, dependency, or non-reconstructability must not be erased as narrative noise.
- **Condition binding**: compressed judgments remain reusable only while their stored conditions remain valid.

This converts the common failure mode—post-hoc narrative replacement—into a constrained update form: the past is played back, the future is edited, and reusable residue is condition-bound.

F. Judgment Compression pipeline

The Judgment Compression pipeline consumes DFR output and emits a reusable gate state only when the residue qualifies for preservation.

Operationally, the pipeline is:

- 1) **DFR extraction**: separate As-of Playback, Delta, Residue, and Next step.
- 2) **Residue filtering**: preserve only residue that affects branching, reversibility, Seat, dependency, or non-reconstructability.
- 3) **Compression**: encode the preserved structure as (outcome, residue_key, condition).
- 4) **Storage**: write the compressed state to the Judgment State Registry.
- 5) **Reuse check**: before reuse, verify that the stored condition still holds.
- 6) **Invalidation fallback**: if the condition is invalidated, stale, or unverified, downshift to DELAY and return to Recheck or DFR.

This pipeline is illustrated in Figure 1. The figure should be read as an evidence diagram rather than a decorative diagram: it shows how DFR separates forward delta from preserved residue, how irreducible mismatch functions only as a residue candidate signal, and how condition invalidation returns the state to DELAY.

G. Release pipeline as an operational gate

Release is treated as a pipeline that consumes a stabilized state and emits a release decision.

a) *Inputs*: The pipeline reads:

- the latest As-of Pack and Seat assignment,
- the current CAP record, including alert and recheck,
- the current severity outcome (PASS, DELAY, or BLOCK),
- reusable judgment states whose conditions remain valid,
- impact magnitude and public exposure signals.

b) *Gate rule*: Release is granted only if:

- the merged outcome is PASS,
- any reused compressed judgment has a valid condition,
- and the continuous pass condition holds over the required observation window, scaled by impact.

If the state is DELAY or BLOCK, release is rejected by design. If a reused judgment state has an invalidated or unverified condition, the release pipeline treats it as DELAY until rechecked. This keeps publication aligned with operational safety rather than narrative confidence.

H. *Auditability: replay, diff, and reuse trace*

V9 supports three audit operations.

a) *Replay (Playback-only)*: Given an As-of Pack and the recorded state deltas, an auditor can reconstruct the exact state at the time of the decision without injecting new facts.

b) *Diff (Forward-only)*: Given a review, the auditor extracts the delta that will be applied going forward. The delta is explicit, minimal, and recorded as a single operational change.

c) *Reuse trace*: Given a compressed judgment, the auditor can trace:

- the source As-of Pack,
- the DFR record,
- the residue criterion that justified preservation,
- the stored (outcome, residue_key, condition) tuple,
- and the condition check used at reuse time.

This architecture turns evaluation from a debate into a reproducible procedure: *replay the past, extract the forward delta, preserve qualifying residue, and reuse only condition-valid gate states*.

V. TIME V2 AND HINDSIGHT CONTROL

This chapter defines Time V2: a time-aware evaluation rule that preserves temporal fidelity and prevents hindsight from rewriting the past.

Time V2 is not a forecasting tool. It is a constraint on explanations and reviews:

- the past is evaluated only under the As-of Pack (Playback-only),
- improvements are written only as forward deltas (Forward-only update),
- preserved residue may be reused only when its stored condition remains valid.

In this update, Time V2 also governs reusable judgment states. A compressed judgment can help future decisions only if its condition still holds; otherwise, the judgment is downshifted to DELAY and returned to recheck or DFR.

A. *Two lanes: playback-only and forward-only*

Time V2 separates evaluation into two lanes.

a) *Playback-only lane*: The reviewer must restate the decision using only the As-of Pack. No new facts, no upgraded capabilities, no revised objectives.

b) *Forward-only update lane*: Any improvement is written as a delta that applies from now on. The delta must include dependencies, alert conditions, and a recheck plan.

This lane separation prevents the most common failure: a review that “wins” by importing future knowledge, and therefore destroys learning.

c) *Residue lane*: V9.1 adds a third storage distinction without creating a third review mode: preserved residue is neither a playback explanation nor a forward delta. It is the structure that remains valid after DFR and passes the Residue Criterion. Only such residue may be compressed into a reusable judgment state.

The residue lane prevents two opposite errors:

- erasing valid structure merely because the outcome changed, and
- turning stale structure into a permanent PASS.

B. *Reverse temporal evaluation: capability ceiling at the time*

Time V2 requires *Reverse temporal evaluation*: past decisions are judged under the capability ceiling and constraints that existed at the time. A reviewer is not allowed to retroactively demand actions that were not feasible, not knowable, or not affordable under the As-of Pack.

Operationally:

- the As-of Pack fixes the feasible action set,
- the review may propose a future delta to expand feasibility,
- preserved residue may be extracted only from what remains valid under the As-of boundary,
- but the evaluation of the past does not borrow future capability or later evidence.

This rule prevents both condemnation by impossible standards and absolution by fabricated constraints.

C. *T_a and D_a: time does not increase value by itself*

This formulation extends the Time-is-an-Ally V3 note, where time amplification is treated as disciplined re-evaluation under external signals rather than automatic validation by elapsed time [4].

Time V2 separates two effects:

- $T_a(t)$: amplification that occurs only when re-evaluation under external signals confirms the structure.
- D_a : degradation or discount that captures decay, drift, or opportunity cost.

A minimal relation is:

$$V_{\text{later}} = T_a(t) \times D_a.$$

Crucially, $T_a(t)$ is not a function of time alone. It increases only through re-evaluation triggered by external signals, such as:

- independent confirmation,
- adversarial testing,
- real-world deployment evidence,
- or repeated survival under public exposure.

This prevents a common narrative abuse: claiming that time automatically validates a decision.

For reusable judgments, the same rule applies: time does not automatically preserve a compressed judgment. A stored judgment remains reusable only while its condition remains valid under current re-evaluation.

D. *Condition validity over time*

Judgment Compression stores preserved residue as:

$$(\text{outcome}, \text{residue_key}, \text{condition}).$$

The *condition* is the temporal boundary of reuse. It specifies when the compressed judgment remains valid and prevents the residue from becoming an unconditional rule.

A condition may fail for at least four reasons:

- **Context drift:** the operating context no longer matches the original reuse scope.
- **Dependency failure:** a dependency recorded in CAP no longer holds.
- **Seat change:** responsibility assignment or decision authority has changed.
- **Evidence update:** new evidence changes the status of the preserved residue.

When the condition is invalidated, stale, or unverified, the compressed judgment does not remain PASS by default. It is downshifted to DELAY and must be rechecked through DFR before reuse.

E. *Re-evaluation triggers and recheck windows*

Time V2 treats re-evaluation as a scheduled operation with explicit triggers. A claim must specify:

- **Alert-if:** signals that invalidate the claim,
- **Recheck:** when the claim will be revisited,
- **Hedge:** what safety posture holds until the recheck.

For compressed judgments, the recheck window also applies to the stored condition. Before reuse, the system must verify that the condition remains valid. If the condition cannot be verified, the default posture is DELAY.

When uncertainty cannot be bounded, the default posture is DELAY with an explicit until time. Time V2 therefore turns “waiting” into a controlled safety state rather than a vague pause.

F. *Misuse prohibitions*

Time V2 explicitly forbids three misuses.

a) *Prediction misuse:* Time V2 must not be used to claim certainty about future outcomes. It is an evaluation constraint, not a prophecy engine.

b) *Hindsight absolution misuse:* Time V2 must not be used to erase responsibility by rewriting the past. Explanations stay within Playback-only. Improvements belong to the forward-only delta lane.

c) *Stale reuse misuse:* Judgment Compression must not be used to turn a past PASS into a permanent permission. A compressed judgment is condition-bound. If its condition no longer holds, the judgment defaults to DELAY and returns to recheck or DFR.

G. Operational summary

Time V2 is executed as a fixed rule set:

- 1) Fix the As-of Pack.
- 2) Run Playback-only evaluation under the capability ceiling at the time.
- 3) If change is needed, write exactly one forward-only delta with alert and recheck.
- 4) If residue remains, preserve only residue that passes the Residue Criterion.
- 5) If residue is compressed, store it as (outcome, residue_key, condition).
- 6) Before reuse, verify the stored condition.
- 7) If the condition is invalidated, stale, or unverified, downshift to DELAY and return to DFR.
- 8) Maintain PASS, DELAY, or BLOCK without narrative drift.

Time V2 makes reviews reproducible: the past is replayed as it was, only the future is edited, and reusable residue remains bound to explicit conditions.

VI. INTERACTION PROTOCOLS

Decision-OS V9 treats interaction itself as a risk surface. Many failures are not caused by a lack of intelligence, but by conversational dynamics: narrative smoothing, role drift, and premature commitment.

This chapter defines interaction rules that preserve the V9 anchors: *As-of*, *Seat*, and *Release*. In this update, interaction also becomes the entry point for identifying residue candidates that may later be filtered and compressed into reusable judgment states.

A. Role fixation: Decision Owner vs. Auditor

V9 requires explicit role fixation during interaction.

- **Decision Owner** holds the decision seat and can commit an action.
- **Auditor** may challenge, test, and propose deltas, but does not take the seat.

Role swapping mid-review is prohibited. This prevents a common failure mode where responsibility drifts into “shared vibes” and learning collapses into blame or absolution.

Operationally, every interaction thread begins by stating:

- the Decision Owner,
- the Auditor,
- and whether the thread is **Playback-only** or **Forward-only update**.

If a reusable judgment state is invoked during interaction, the thread must also state:

- the referenced **residue_key**,
- the stored condition,
- and whether the condition currently holds.

B. As-of constrained prompting

When an assistant model is used, V9 constrains prompts to the As-of Pack (Section III-B). The assistant must be instructed to:

- restate the situation using only the provided As-of Pack,
- avoid importing “likely” facts not present in the pack,
- explicitly mark unknowns as unknown, without filling gaps by narrative,
- and distinguish forward deltas from preserved residue.

A minimal As-of prompt contract is:

Use only the As-of Pack below. Do not add new facts. If information is missing, keep it missing. Return (i) Playback-only restatement, (ii) uncertainties, (iii) PASS/DELAY/BLOCK suggestion, and (iv) any residue candidate that should not be edited away.

This contract prevents the assistant from producing convincing but ungrounded reconstructions that overwrite the past. It also prevents residue candidates from being smoothed into narrative coherence before DFR can evaluate them.

C. Conversation drift: narrative smoothing as a failure mode

Assistants often “help” by smoothing contradictions into a coherent story. In high-stakes contexts, this is a safety failure.

V9 therefore forbids three interaction behaviors in Playback-only lanes:

- **Gap filling:** inventing missing premises to complete a story.
- **Moralization:** turning review into character judgment.
- **Outcome worship:** judging past actions primarily by eventual results rather than As-of constraints.

Instead, Playback-only interaction is strictly descriptive:

- what was known,
- what was constrained,
- what was optimized,
- and what was executed.

When contradictions remain after this descriptive pass, they must not be automatically erased. They may indicate residue candidates, especially when the contradiction changes branching, reversibility, Seat assignment, dependency structure, or non-reconstructability.

D. Multi-Model Collision Review (MMAR)

V9 supports Multi-Model Collision Review (MMAR) as an interaction method to extract structure from disagreement.

a) Purpose: When one model produces a highly coherent narrative, it can override uncertainty. MMAR introduces controlled disagreement to surface invariants, failure modes, and residue candidates.

MMAR is not a voting mechanism. It is an interaction method for detecting structure that survives disagreement without collapsing uncertainty into persuasive coherence.

b) Independence rule: Models must not share conversational memory. Each model receives the same As-of Pack, but no cross-context.

c) Minimal MMAR prompt set: For each model k , request:

- the strongest opposing argument, namely the best counter-case,
- likely failure modes and their detection signals (Alert-if),
- a PASS/DELAY/BLOCK recommendation with reasons tied to the As-of Pack,
- and any residue candidate that should be preserved rather than smoothed away.

d) Merge rule: MMAR outputs are merged conservatively:

- outcome severity uses max over PASS/DELAY/BLOCK,
- evidence is unioned,
- proposed updates are queued as forward deltas under the one-change rule in Section III-E,
- and residue candidates are sent to the Residue Criterion rather than directly stored.

MMAR is not used to “vote for truth”. It is used to detect structural risk, preserve disagreement signals, and prevent narrative overwrite.

E. Residue candidates from interaction

Interaction may generate residue candidates in three common ways.

a) Model disagreement: Different models identify incompatible failure modes, dependency structures, or Seat implications under the same As-of Pack.

b) Irreducible mismatch: A reconstructed explanation fails to recover the original structure. This mismatch is not treated as a probability estimate or proof of correctness. It is treated only as a candidate signal that the missing structure may be non-reconstructable.

c) Repeated recheck friction: A claim repeatedly returns to DELAY because its condition, dependency, or evidence cannot be verified.

These candidates are not automatically preserved. They must pass the Residue Criterion: the candidate must affect branching, reversibility, Seat assignment, dependency structure, or non-reconstructability. Only then may it be entered into the Residue Registry and considered for Judgment Compression.

F. Interaction guardrails: stopping, delaying, rolling back

V9 makes three guard actions explicit in interaction.

a) Stop: If the action is unsafe or the seat is undefined, interaction must halt in **BLOCK** until corrected.

b) *Delay*: If uncertainty is high and impact is high, interaction defaults to **DELAY** with an explicit until time and recheck triggers (Section III-I).

If a compressed judgment is invoked but its stored condition is invalidated, stale, or unverified, interaction also defaults to **DELAY**. The prior PASS is not reused until DFR or recheck confirms that the condition holds.

c) *Rollback*: If a decision has already been executed and new evidence appears, the interaction must:

- 1) replay the original As-of Pack (Playback-only),
- 2) then propose exactly one forward delta (Forward-only update),
- 3) preserve any residue candidate separately from the delta,
- 4) and specify a hedge action during the transition (CAP Hedge).

These guardrails keep interaction procedural rather than persuasive.

G. Output contracts: CAP + DFR + Judgment Compression

To prevent drift, V9 treats CAP and DFR not as documentation, but as *interaction primitives*. This update adds Judgment Compression as a reuse primitive, but only after DFR and residue filtering.

a) *CAP as commitment boundary*: Before committing to an action, interaction must be able to state: Claim, Dependencies, Alert-if, Recheck, Hedge (Section III-D).

b) *DFR as review boundary*: After an action, interaction must produce: As-of playback, Delta, Residue, Next step (Section III-E).

c) *Judgment Compression as reuse boundary*: Before a preserved residue can be reused, interaction must produce:
(outcome, residue_key, condition)

and verify that the condition currently holds.

If the condition does not hold, the output must be DELAY, not PASS. This prevents interaction from turning old conclusions into hidden permanent permissions.

By enforcing these outputs, V9 converts conversation into reproducible state transitions.

H. Summary

V9 interaction is constrained by:

- **Role fixation** (Seat does not drift),
- **As-of constrained prompting** (Playback-only is factual),
- **MMAR** for structure extraction (disagreement reveals invariants and residue candidates),
- **Residue Criterion** before preservation,
- **Guardrails** (Stop / Delay / Rollback),
- **CAP + DFR** as mandatory interaction outputs,
- and **Judgment Compression** only for condition-bound reuse.

The goal is not to sound convincing. The goal is to remain correct *as a procedure* under public pressure, hindsight pressure, and repeated reuse.

VII. MULTI-PARTY CONFIRMATION AND SEAT CONTROL

High-impact decisions often fail not because the decision owner is irrational, but because confirmation pathways are weak. Under pressure, a single actor can be nudged into irreversible actions by narrative momentum, interface friction, or social coercion.

This chapter defines a minimal multi-party confirmation layer that preserves the V9 anchors: *Seat* (decision rights), *As-of* (temporal fidelity), and *Release* (impact-weighted gating).

In this update, Seat control also applies to reusable judgment states. A compressed judgment may support a future decision only when its condition remains valid and the current Decision Owner retains the seat.

A. Why multi-party confirmation is needed

Single-party confirmation is fragile under:

- high-stakes urgency (deadline compression),
- asymmetric information (the user cannot verify claims),
- repeated persuasion attempts (Public Tube pressure),
- and stale reuse of prior judgments whose original conditions no longer hold.

Multi-party confirmation introduces a deliberate speed bump that:

- increases detection probability for scams and coercion,
- reduces impulsive irreversible actions,
- creates an auditable trail for later DFR updates,
- and prevents compressed judgments from becoming hidden permanent permissions.

B. Seat control: decision rights are not negotiable mid-stream

V9 treats Seat as a pre-evaluation gate. Seat determines who can commit actions and who can audit them.

- The **Decision Owner** may execute actions.
- The **Auditor** may challenge and propose deltas.

During high-impact flows, V9 strengthens Seat control by requiring external confirmation before execution. This prevents role drift and reduces the chance that persuasion converts into commitment.

Seat control also constrains reuse. A reusable judgment state does not transfer decision rights. Even if a compressed judgment has outcome PASS, the current Decision Owner must still verify that the stored condition holds before the judgment can support action.

C. Confirmation states

V9 defines a minimal confirmation state machine:

- **CONFIRM-0** (Solo): Decision Owner confirms without external party.
- **CONFIRM-1** (Buddy): One external confirmer is required.
- **CONFIRM-2** (Family / Team): Two confirmers are required for irreversible actions.

The required level is impact-weighted: higher impact requires higher confirmation level.

For compressed judgments, the confirmation level is determined by the current action, not by the historical action that produced the residue. A prior low-impact residue cannot automatically authorize a later high-impact use.

D. What must be confirmed

Confirmation is not “approval of intent.” It is a structured check of the As-of Pack, the CAP record, and any reused judgment state.

A confirmer must verify:

- **Action:** what will be executed exactly.
- **Destination:** who receives value or access, such as recipient, address, endpoint, audience, or publication surface.
- **Irreversibility:** whether rollback is possible.
- **CAP fields:** Dependencies, Alert-if, Recheck, Hedge.
- **Seat:** who currently holds the Decision Owner role.
- **Condition validity:** whether any reused (outcome, residue_key, condition) still holds.

If any of these cannot be verified, the default state is **DELAY**. In particular, if the stored condition of a compressed judgment is invalidated, stale, or unverified, the judgment cannot be reused as PASS.

E. Speed control: delay as a protective valve

A key purpose of multi-party confirmation is to restore time. V9 treats time not as a luxury but as a safety resource.

For high-impact actions, confirmation enforces:

- an explicit *until* window (cooldown),
- a recheck step (independent verification),
- and a hedge posture while waiting, such as freeze, partial rollback, downshift, or no-action.

This converts “wait” into a controlled protocol rather than hesitation.

When a reusable judgment state is invoked under time pressure, speed control is stricter: the system must first verify the condition, then confirm the current Seat, and only then decide whether PASS remains available. If either check fails, the state is DELAY.

F. Audit trail: evidence union and replayability

Multi-party confirmation must be auditable.

a) *Evidence union:* All confirmers append evidence. Evidence is merged as a set union and cannot be deleted by later narrative explanations.

b) *Replay:* A later audit can replay:

- the As-of Pack,
- the CAP record,
- the confirmation state and confirmations,
- the executed action,
- any reused residue key,
- and the condition check used at reuse time.

This supports DFR: the past is replayed as it was; only the future is edited. It also supports Judgment Compression audit: a reused judgment must be traceable back to the residue that produced it and the condition that allowed reuse.

G. Failure modes and defenses

Multi-party confirmation is not a guarantee. V9 therefore models common failure modes:

- **Collusion:** confirmers are not independent.
- **Capture:** confirmers are pressured or socially coerced.
- **Rubber-stamp:** confirmation becomes a ritual with no verification.
- **Stale reuse:** a previously compressed judgment is reused after its condition no longer holds.
- **Seat drift:** a confirmer or assistant implicitly becomes the decision owner without explicit delegation.

Defenses are procedural:

- diversify confirmers when possible (independence),
- require at least one external verification step (recheck),
- verify the current condition before reusing any compressed judgment,
- preserve the current Decision Owner explicitly,
- and treat uncertainty as DELAY, not as permission to proceed.

H. Summary

V9 multi-party confirmation is a minimal protection layer:

- impact-weighted confirmation states (CONFIRM-0/1/2),
- structured verification of action, destination, irreversibility, CAP fields, Seat, and condition validity,
- enforced cooldown and recheck windows (DELAY as protective valve),
- append-only audit trail (evidence union, replayability),
- and condition-bound reuse of compressed judgments.

This layer preserves Seat control under pressure and reduces irreversible mistakes under public repetition. It also prevents reusable judgment states from bypassing present-time confirmation.

DISCLOSURE

- Scope and intent.* This work proposes an operational decision protocol (Decision-OS V9) for preserving temporal fidelity, responsibility assignment, reusable judgment states, and safe release under public repetition. It is not a prediction system and does not claim empirical guarantees.
- Safety posture.* The protocol reduces harm by enforcing procedural constraints (PASS/DELAY/BLOCK, As-of Pack fixation, Seat fixation, condition-bound Judgment Compression, and impact-weighted continuous pass). If an action is high-impact, irreversible, cannot be verified under the As-of Pack, or relies on a compressed judgment whose condition is invalidated or unverified, the recommended default posture is **DELAY** or **BLOCK**.
- Judgment Compression boundary.* Judgment Compression does not rewrite the original As-of Pack and does not convert past outcomes into permanent permissions. A compressed judgment is stored only as

(outcome, residue_key, condition)

and remains reusable only while the stored condition holds. If the condition is invalidated, stale, or unverified, the judgment defaults to **DELAY** and must be rechecked through DFR before reuse.

- Residue boundary.* Residue preservation is limited to structure that affects branching, reversibility, Seat assignment, dependency structure, or non-reconstructability. Irreducible mismatch may serve as a residue candidate signal, but it is not a probability estimate, proof of correctness, or standalone release criterion.
- Limitations.* This paper is conceptual and procedural. It does not provide a complete empirical evaluation, formal proofs for all claims, or a comprehensive threat model. Real-world deployment requires domain-specific adaptation, independent auditing, condition validation, and continuous re-evaluation.
- Ethical considerations.* The protocol aims to prevent hindsight-based responsibility laundering, scapegoating, narrative overwrite, and stale reuse of prior judgments. Reviews should focus on As-of constraints, preserved residue, and forward deltas, not on character judgment. High-risk use cases (e.g., medical, legal, financial, and safety-critical operations) should require stricter confirmation and independent oversight.
- Conflicts of interest.* The author declares no external conflicts of interest.

REFERENCES

- [1] B. Fischhoff, "Hindsight $\neq$ foresight: The effect of outcome knowledge on judgment under uncertainty," *Journal of Experimental Psychology: Human Perception and Performance*, vol. 1, no. 3, pp. 288–299, 1975.
- [2] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, "Model cards for model reporting," 2018. [Online]. Available: <https://arxiv.org/abs/1810.03993>

- [3] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” U.S. Department of Commerce, National Institute of Standards and Technology, Tech. Rep. NIST AI 100-1, Jan. 2023. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>
- [4] S. Nagata, “Decision-OS Note: Time-is-an-Ally V3 — Value as a Non-Decreasing Trajectory Under External Signals,” Mar. 2026, preprint. [Online]. Available: <https://doi.org/10.5281/zenodo.19076241>
- [5] —, “Decision-OS V5 Revised: SiriusA — A Zero-Knowledge Confirmation Layer for Trajectory-Aware Human–AI Decision Safety,” 2026, preprint. Genesis DOI: 10.5281/zenodo.17480645.
- [6] —, “Decision-OS V6: Canonical Commitment Architecture for Non-Destructive Integration,” 2026, preprint.
- [7] —, “Decision-OS V7: Aspire Intelligence for AGI — Operational Hook: Per-Update Gate with Single-Window Evaluation,” 2026, preprint.
- [8] —, “Decision-OS V8: Time-Tube Control for Self-Safe AGI — Trajectory Control under Irreducible Mismatch,” 2026, preprint.
- [9] —, “Decision Constitution v1.1,” 2026, operational constitution for temporal fidelity, As-of responsibility fixation, and forward-only change.